

slice

– это динамический массив, но совсем не массив. – это дескриптор(описатель) участка памяти.

Устройство

```
type slice struct {
    array unsafe.Pointer // Указатель на первый элемент массива (underlying array)
    len  int             // Длина слайса (количество элементов, доступных для
использования)
    cap  int             // Емкость слайса (максимальное количество элементов до
переаллокации)
}
```

Важные нюансы:

- Слайс не владеет данными. Он всегда ссылается на базовый массив (underlying array), который находится в куче (heap) или статической памяти.
- При передаче слайса в функцию копируется только эта структура (24 байта на 64-битной системе: 8 байт указатель + 8 len + 8 cap), а не все данные. Это делает передачу слайсов очень эффективной.

Создание

Через литерал (созд. массив и ссылку на него)

```
s1 := []int{1, 2, 3} // len=3, cap=3
// не путать с [...]int{1, 2, 3}
```

Через make (рекомендуемый для динамических данных)

```
s2 := make([]int, 5) // len=5, cap=5 (все элементы равны нулю)
s3 := make([]int, 0, 10) // len=0, cap=10 (память выделена, но элементов нет)
```

С помощью new (редко, возвращает указатель):

```
s4 := new([]int) // указатель на nil-слайс
Nil-слайс (без памяти):
var s5 []int // nil, len=0, cap=0, указатель == nil
Пустой слайс (с памятью, но без элементов):
s6 := []int{} // не nil, len=0, cap=0, указатель не nil (указывает на zero-sized
адрес)
```

Разница между nil и пустым слайсом критична при сериализации в JSON: nil превратится в null, а пустой – в [].

slicing

```
base := []int{0, 1, 2, 3, 4, 5}
sub := base[1:4] // sub = [1, 2, 3], len=3, cap=5 (от индекса 1 до конца массива)
```

append

Переаллокация происходит если len == cap, тогда создается новый массив, в 2 раза больше старого (до 1024 эл) или в 1.25 больше (больше 1024 эл), и слайс будет ссылаться на него.

Если два слайса шарят один массив, и вы делаете append к одному из них, вызвав переаллокацию, второй слайс продолжит указывать на старый массив, и они “разъедутся” (потому что каждый из слайсов будет ссылаться на свой отдельный массив).

При этом старые элементы копируются в новый массив из старого

Полный список необходимых методов

Функция	Назначение	Важное предостережение
<code>len(s)</code>	Возвращает длину (количество элементов).	Сложность $O(1)$ (берет из структуры).
<code>cap(s)</code>	Возвращает емкость.	Сложность $O(1)$ (берет из структуры).
<code>append(s, elem)</code>	Добавляет элемент(ы) в конец.	Может создать новый массив. Работает даже с nil слайсом.
<code>copy(dst, src)</code>	Копирует элементы из src в dst.	Возвращает число скопированных элементов. Важно: выделяет память под dst нужно заранее (делать make).
<code>s = append(s[:i], append(s[i+1:], elem)...) s = append(s[:i], append([]T{elem}, s[i:]...)...)</code>	Удаление элемента по индексу i. Вставка элемента в середину.	Хак с нарезкой, сохраняет порядок. Сложная, но эффективная.

Оператор распаковки ...

Пример:

```
s = append(s[:i], s[i+1:]...)
```

Но так нельзя:

```
s := []string{"A", "B", "C"}
fmt.Println(s...) // Ошибка: cannot use s (variable of type []string) as type []any
in argument to fmt.Println
```

А так можно:

```
s := []string{"A", "B", "C"}

// Превращаем []string в []any
anySlice := make([]any, len(s))
for i, v := range s {
    anySlice[i] = v
}
```

```
fmt.Println(anySlice...) // Вывод: A B C
```

Почему же?

Рассмотрим функцию `fmt.Println()`

```
func Println(a ...any) (n int, err error)
```

Она принимает вариативный параметр `...any`, то есть любое количество аргументов причем любого типа.

Но есть правило:

- Срез можно распаковывать только при передаче в вариативный параметр, но при этом тип элементов среза должен совпадать с типом вариативного параметра

A any != string.

Утечка памяти

```
big := make([]byte, 1_000_000)
small := big[:2]
```

Теперь small указывает на массив на 1 млн элементов. Сборщик мусора (GC) не очистит big, пока small жив. Это утечка.

Решение: Если нужны только первые 2 элемента, скопируйте их в новый слайс:

```
small := make([]byte, 2)
copy(small, big[:2]) // big теперь может быть удален GC.
```